

UNIT I: PROCESS MANAGEMENT & CPU SCHEDULING ALGORITHMS

Weightage: 25% | Syllabus Section 1

1. Process States & Context Switching

Process Control Block (PCB): PID, Program Counter, Registers, Scheduling Info, Memory Limits, Open File Descriptors.
5-State Lifecycle: New → Ready → Running → Waiting/Blocked → Terminated.
Context Switch: Saving state of active process and restoring state of scheduled process. Pure overhead: $T_{cs} \approx 1 - 10 \mu s$.

2. CPU Scheduling Metrics & Formulas

Turnaround Time (TAT): $TAT = Completion\ Time\ (CT) - Arrival\ Time\ (AT)$.
Waiting Time (WT): $WT = TAT - Burst\ Time\ (BT)$.
Response Time (RT): Time from arrival to first CPU allocation.
Throughput: Total processes completed per unit time.

3. Classic Scheduling Algorithms

FCFS: Non-preemptive. Suffers from Convoy Effect (short processes delayed behind long CPU-bound process).
SJF / SRTF: Shortest Job First. Provably optimal minimum average waiting time! Preemptive version is Shortest Remaining Time First (SRTF).
Round Robin (RR): Preemptive with time quantum q . If $q \rightarrow \infty \implies FCFS$; if $q \rightarrow 0 \implies$ Processor Sharing.

UNIT II: PROCESS SYNCHRONIZATION & DEADLOCK AVOIDANCE

Weightage: 25% | Syllabus Section 2

1. Critical Section & Synchronization

3 Core Requirements: (1) Mutual Exclusion, (2) Progress, (3) Bounded Waiting.
Semaphores: Integer variable accessed via atomic primitives:
• $P(S) / wait(S): while(S \leq 0); S = S - 1;$
• $V(S) / signal(S): S = S + 1.$
Classic Problems: Producer-Consumer (Bounded Buffer), Reader-Writer (Semaphores mutex & wrt), Dining Philosophers.

2. Deadlock Characterization

Coffman's 4 Necessary Conditions:
1. Mutual Exclusion (non-shareable resources)
2. Hold and Wait
3. No Preemption
4. Circular Wait (cycle in Resource Allocation Graph)
Deadlock Prevention: Invalidate at least one of the 4 conditions (e.g., total resource ordering eliminates circular wait).

3. Banker's Algorithm (Avoidance)

Data Structures: $Allocation[n][m], Max[n][m], Available[m]$.
Need Matrix: $Need[i][j] = Max[i][j] - Allocation[i][j]$.
Safety Algorithm: Find process i where $Need_i \leq Work$. Update $Work = Work + Allocation_i$. If all finish, system is in **Safe State**; no deadlock!

UNIT III: MEMORY MANAGEMENT & VIRTUAL MEMORY PAGING

Weightage: 25% | Syllabus Section 3

1. Paging Architecture & Address Translation

Logical Address: (p, d) , where p is page number, d is page offset.
Physical Address: (f, d) , where f is frame number from Page Table.
Memory Calculation: If page size is 2^k bytes, offset d requires k bits. For 32-bit address and 4KB page size (2^{12}), $p = 20$ bits, $d = 12$ bits.

2. TLB & Effective Memory Access Time

Translation Lookaside Buffer (TLB): High-speed associative hardware cache for page table entries.
EMAT Formula: $EMAT = h(t_{TLB} + t_m) + (1 - h)(t_{TLB} + 2t_m)$, where h is TLB hit ratio, t_m is memory access time.
Two-Level Paging: EMAT without TLB requires 3 memory accesses!

3. Page Replacement & Thrashing

Page Fault: Trap triggered when referenced page valid bit is 0 (not in RAM). Serviced via disk I/O.
Algorithms:

- FIFO: Replaces oldest page. Suffers from **Belady's Anomaly** (more frames $\implies$ more page faults)!
- LRU: Replaces least recently used page. Stack algorithm; immune to Belady's anomaly.
- Optimal: Replaces page not used for longest future duration. Theoretical benchmark!

Thrashing: When CPU spends more time paging than executing: $\sum WorkingSet_i > Total\ RAM$.

UNIT IV: STORAGE MANAGEMENT, DISK SCHEDULING & FILE SYSTEMS

Weightage: 25% | Syllabus Section 4

1. Disk Scheduling Algorithms

Seek Time: Dominant delay in magnetic disk access. Arm moves across cylinders.
Algorithms:

- FCFS: Fair but high total head movement.
- SSTF (Shortest Seek Time First): Selects request closest to head. High throughput, but starvation possible for distant tracks!
- SCAN (Elevator): Arm moves in one direction to end, servicing requests, then reverses.
- C-SCAN (Circular SCAN): Serves in one direction only; jumps back to start without serving return requests (more uniform wait times!).
- LOOK / C-LOOK: Like SCAN but only goes as far as last request in direction.

2. File Allocation & UNIX Inode Architecture

File Allocation Methods:

- Contiguous: Fast sequential access, suffers from external fragmentation.
- Linked: No fragmentation, slow random access, pointer overhead.
- Indexed: Index block holds pointers. Efficient direct access.

UNIX Inode Structure: 12 Direct blocks, 1 Single indirect, 1 Double indirect, 1 Triple indirect pointer. Supports multi-gigabyte file addressing with fast direct access for small files!